

# Builder Journal

## Week 3 — Helmet Detection & Tracking Analytics

### Biggest Challenge

The hardest part of this week wasn't writing the tracking code itself — it was diagnosing why the numbers it produced were wrong. Early runs showed Total Objects climbing steadily to 27+ while Active Objects stayed under 8, with individual tracks lasting only 6-15 frames in a scene where people were visibly present the whole time. Distinguishing 'the tracker is buggy' from 'the tracker is behaving correctly given a hard scene and an imperfect detector' took real, evidence-based debugging rather than guessing at fixes.

### Tracking Issues Encountered

- ID churn / track fragmentation: the same physical objects were repeatedly assigned new tracker IDs instead of maintaining one continuous identity, inflating the Total Objects count far beyond the true number of people in frame.
- Single-frame 'ghost tracks': several tracks appeared for exactly one frame at high confidence (0.72-0.93) and then vanished. Traced this to supervision's default `minimum_consecutive_frames=1`, which confirms a track from a single detection with no persistence requirement.
- A static BoT-SORT config file silently ignored the app's confidence/buffer sliders, meaning an entire round of BoT-SORT testing was accidentally run with unchanged settings — caught only by explicitly logging the live, active tracker configuration before each run.
- Line-crossing (entered/exited) never registered a single count across any configuration tested, consistent with tracks never staying stable long enough to complete a full crossing.
- Repeated identical-configuration runs of BoT-SORT produced contradicting results between attempts (34 vs 22 unique objects, 6 vs 1 ghost tracks) — the pipeline is not fully deterministic, which limits how much confidence can be placed in any single comparison run.

### Performance Bottlenecks

- BoT-SORT ran at roughly half the FPS of ByteTrack (2.5-3.4 vs 3.5-4.2 fps) due to its added camera-motion compensation and appearance-based re-identification computation.
- Training run 3 (yolov8s) took 46 hours instead of an expected ~3.5 hours, almost certainly due to the laptop sleeping or thermal throttling mid-run — a reminder to keep the machine awake and cool during long unattended jobs.
- Resizing input frames to lower resolutions (320/640/1280) did not produce the expected FPS gains, and in one case ran slower than native 2160px resolution.

### Optimization Techniques Applied

- Raised `minimum_consecutive_frames` from 1 to 3, requiring a detection to persist across several frames before being confirmed as a real track, directly reducing ghost tracks.
- Made the BoT-SORT config file dynamic (written per-run from the actual UI slider values) instead of a static file with hardcoded values, so every configuration change actually takes effect.
- Lowered the default confidence threshold from 0.45 to 0.30 after finding real detections sitting at 0.46-0.59 confidence — right at the old cutoff — which caused them to flicker in and out of existence frame to frame.
- Added a live 'active configuration' panel in the app so any tracker/threshold/buffer setting can be visually confirmed before processing, rather than assumed.

### Lessons Learned

- A detector's known weaknesses (small-object recall, background false positives) don't disappear in a tracking context — they resurface as fragmentation, ghost tracks, and failed line crossings. Tracking-layer tuning can shift how imperfections manifest but cannot fully compensate for a weaker base detector.
- Always verify that a configuration change actually took effect before concluding it didn't work. A static config file silently overriding a UI slider produced a completely misleading 'nothing changed' result until the active config was explicitly logged and inspected.

- Single-run comparisons between algorithms (e.g. ByteTrack vs BoT-SORT) can be unreliable if the pipeline has any non-determinism; an honest report says so rather than picking whichever run looked better.
- Ultralytics' YOLO internally resizes every frame to its own fixed inference size regardless of input resolution — so pre-resizing frames in application code is not a dependable way to gain speed.

## Week 4 Goals

- Run each tracker configuration multiple times and average results to get a statistically meaningful ByteTrack vs BoT-SORT comparison instead of a single noisy run.
- Collect and label a small set of ground-truth tracks (manual person-by-person annotation across frames) to compute real tracking metrics (ID switches, fragmentation ratio) rather than proxy metrics alone.
- Investigate why the counting line never registers a crossing — test with a lower-density, less-occluded clip to isolate whether it is purely a fragmentation side-effect or a separate line-placement issue.
- Expand hard-negative background examples for the base detector at greater scale (hundreds rather than dozens), since this remains the most likely lever for improving both detection and downstream tracking quality.